Implementation and Evaluation of FastSpeech 2 for English and Telugu Text-to-Speech Synthesis

1 Abstract

Text-to-Speech (TTS) synthesis is an important application of artificial intelligence and speech processing that enables the conversion of written text into natural-sounding human speech. Recent developments in deep learning have significantly improved neural speech synthesis systems in terms of speech quality, generation speed, and naturalness. Among these approaches, FastSpeech 2 is a non-autoregressive neural TTS architecture that generates speech efficiently by incorporating explicit representations of speech duration, pitch, and energy.

This project focuses on the implementation and evaluation of the FastSpeech 2 architecture for two languages: English and Telugu. The objective is to investigate the application of a common neural speech synthesis framework to languages with different linguistic characteristics and writing systems. The English experiments use the Svarah speech dataset, while the Telugu experiments use the IndicTTS Telugu dataset. Both datasets undergo preprocessing to prepare the textual and acoustic data required by the model. The preparation process includes audio processing, text representation, speech alignment, and the extraction of acoustic features used during training.

The implementation is based on an existing open-source FastSpeech 2 repository and the architecture described in the original research paper. The English implementation uses the existing text-processing pipeline, whereas the Telugu implementation requires language-specific modifications to support Telugu Unicode characters. These modifications include extending the text symbol representation and adapting the text-processing pipeline to handle Telugu input while maintaining compatibility with the original English functionality. The two language experiments are configured and handled separately to accommodate their respective datasets and text representations.

The experimental study examines the model's ability to learn the relationship between textual input and acoustic speech representations. Particular attention is given to the variance adaptor, which predicts phoneme-level duration, pitch, and energy information. Duration predictions are used to determine the temporal allocation of acoustic frames, while pitch and energy predictions represent important prosodic characteristics of speech. The generated outputs are examined using prescribed test sentences, synthesized audio samples, training observations, and visual analyses of the predicted acoustic features.

The report documents the theoretical foundations of FastSpeech 2, dataset preparation procedures, language- specific implementation changes, experimental configurations, and the observations obtained from the English and Telugu experiments. It also discusses

the technical challenges involved in extending an existing neural TTS implementation to support Telugu and identifies possible improvements for multilingual and low-resource speech synthesis systems.

Keywords: FastSpeech 2, Text-to-Speech Synthesis, Neural Speech Synthesis, English, Telugu, Svarah Dataset, IndicTTS, Variance Adaptor, Duration Prediction, Pitch Prediction, Energy Prediction, Mel-Spectrogram.

2 Introduction

2.1 Background

Speech is one of the most natural and widely used forms of human communication. With the increasing integration of artificial intelligence into everyday applications, enabling computers to understand and generate human speech has become an important area of research. Speech processing technologies are now used in virtual assistants, automated customer support systems, navigation applications, accessibility tools, educational platforms, and human-computer interaction systems.

Text-to-Speech (TTS) synthesis is a fundamental technology within the field of speech processing. It refers to the automatic conversion of written text into spoken language using computational models. A TTS system receives textual input and produces a corresponding speech waveform that represents how the input text would be spoken by a human speaker.

The primary objective of a TTS system is not merely to convert individual words into sounds, but also to generate speech that is intelligible, natural-sounding, and linguistically appropriate. This requires the system to capture several characteristics of human speech, including pronunciation, speaking rate, pauses, pitch variations, intonation, and energy.

The development of TTS systems has progressed from rule-based and concatenative approaches to statistical models and modern deep learning-based architectures. Traditional systems relied heavily on manually designed linguistic rules or prerecorded speech segments. Although these methods established the foundation for speech synthesis, they presented limitations in terms of flexibility, naturalness, and scalability.

Recent advances in deep learning have enabled neural networks to learn complex relationships between textual representations and acoustic speech features directly from training data. These developments have significantly influenced modern TTS research and have led to architectures capable of generating high-quality speech with improved efficiency.

2.2 Evolution of Text-to-Speech Synthesis

Text-to-Speech synthesis has evolved from traditional concatenative and statistical approaches to modern deep learning-based systems. Early concatenative methods generated speech by combining prerecorded audio segments, while statistical parametric approaches used models such as Hidden Markov Models (HMMs) to predict acoustic characteristics. Although these techniques provided the foundation for speech synthesis, they had limitations in naturalness and flexibility. With the advancement of deep learning, neural network-based architectures such as Tacotron and Tacotron 2 improved the quality of synthesized speech by learning complex relationships between text and acoustic features. Further developments introduced non-autoregressive architectures such as FastSpeech and FastSpeech 2, which aim to improve speech generation efficiency while maintaining synthesis quality.

2.3 Introduction to FastSpeech 2

FastSpeech 2 is a non-autoregressive neural Text-to-Speech architecture proposed to generate high-quality speech with improved synthesis efficiency. It addresses limitations associated with earlier autoregressive approaches and introduces explicit modeling of important variations in speech.

In an autoregressive TTS model, acoustic frames are typically generated sequentially, with each prediction depending on previously generated frames. This creates a dependency between successive predictions and can increase the time required for speech generation.

FastSpeech 2 adopts a non-autoregressive generation strategy in which acoustic representations are generated in parallel after the required input representations and sequence lengths have been determined.

An important characteristic of FastSpeech 2 is its use of additional speech information during training. The model incorporates duration, pitch, and energy representations through a component called the *Variance Adaptor*.

These features help the model represent variations in speaking rate, intonation, and loudness, which are important for producing natural-sounding speech.

The architecture consists of the following major components:

- **Text Encoder:** Processes the input text or phoneme sequence and converts it into hidden representations containing linguistic information.
- **Variance Adaptor:** Incorporates duration, pitch, and energy information into the encoder representations to model variations in speech.
- **Length Regulator:** Uses predicted or reference duration information to expand the encoder sequence to the required acoustic frame length.

- **Mel-Spectrogram Decoder:** Converts the expanded hidden representations into acoustic mel-spectrogram features.
- **Vocoder:** Converts the predicted mel-spectrogram into a time-domain audio waveform.

The overall architecture and the individual components are discussed in greater detail in the Methodology section of this report.

2.4 Importance of Duration, Pitch, and Energy Modeling

Human speech contains considerable variation even when the same sentence is spoken by the same speaker. Words and phonemes differ in their duration, pitch, and energy depending on pronunciation, emphasis, speaking rate, and linguistic context.

A speech synthesis model must learn these variations to generate speech that sounds natural rather than producing a uniform sequence of acoustic features.

FastSpeech 2 explicitly incorporates three important types of information through its variance adaptor.

2.4.1 Duration

Duration represents the amount of time associated with individual phonemes or input tokens during speech production. Different phonemes require different amounts of time for articulation.

For example, a long vowel generally occupies more time than a short vowel, while consonants and silences may have different temporal characteristics.

The duration predictor estimates the number of acoustic frames associated with each input unit. These predictions are used by the length regulator to expand the encoder representations into a sequence suitable for acoustic decoding.

Accurate duration modeling is important for maintaining appropriate speaking rates and preserving the temporal structure of synthesized speech.

2.4.2 Pitch

Pitch is an important perceptual characteristic of speech that is closely related to the fundamental frequency of vocal-fold vibration.

Variations in pitch contribute to intonation, stress, emphasis, and the expressive characteristics of spoken language.

In FastSpeech 2, pitch information is modeled through pitch-related acoustic features. The pitch predictor estimates these characteristics, which are subsequently incorporated into the hidden representations of the model.

Analyzing pitch predictions helps in understanding how the model captures variations in the fundamental frequency of speech.

2.4.3 Energy

Energy represents the strength or intensity of the speech signal. It is associated with variations in loudness and emphasis during speech production.

Different phonemes and syllables may exhibit different energy levels depending on their articulatory and acoustic characteristics.

FastSpeech 2 incorporates energy information through an energy predictor and energy embeddings. These representations provide additional information about the intensity characteristics of the generated speech.

The analysis of energy predictions helps examine how the model represents variations in speech intensity across different input sequences.

2.5 Multilingual Speech Synthesis

Although many neural TTS architectures are initially developed and evaluated using a particular language, extending them to other languages introduces additional challenges.

Languages differ in their writing systems, phonetic inventories, pronunciation rules, syllable structures, and linguistic characteristics. Consequently, a text-processing pipeline designed for one language cannot always be directly applied to another language without suitable modifications.

Multilingual and cross-lingual speech synthesis therefore require appropriate handling of language-specific text representations and acoustic characteristics.

The selection of a suitable text representation is particularly important. A model must be able to convert input text into a sequence of representations that can be processed consistently during training and inference.

The ability to support different writing systems is also important for extending speech technologies to languages that have comparatively fewer readily available speech-processing resources.

2.6 Motivation for English and Telugu Implementation

This project investigates the implementation of FastSpeech 2 for English and Telugu. The two languages provide an opportunity to examine the application of the same underlying TTS architecture to different linguistic and writing-system requirements.

English uses the Latin alphabet, whereas Telugu uses its own script, consisting of independent vowels, consonants, vowel signs, and other Unicode characters. Telugu text

processing therefore requires careful handling of Unicode characters and language-specific symbol representations.

The existing English-oriented implementation provides a foundation for the English experiment. However, supporting Telugu requires modifications to the text-processing components so that Telugu input can be represented correctly and converted into sequences that can be processed by the model.

Another motivation is to examine the role of the variance adaptor in the two language experiments. Since duration, pitch, and energy are important characteristics of speech, studying their predicted values provides additional insight into the behavior of the model beyond listening to the generated audio.

By conducting separate experiments for English and Telugu, the project aims to document the implementation requirements, language-specific modifications, and experimental observations associated with applying FastSpeech 2 to both languages.

2.7 Problem Statement

The primary problem addressed in this project is the implementation and evaluation of an existing FastSpeech 2 neural speech synthesis framework for English and Telugu while ensuring that the language-specific text representations and data preparation procedures are handled appropriately.

The original implementation provides an established framework for neural speech synthesis, but adapting it to Telugu requires additional support for Telugu Unicode symbols and appropriate modifications to the text processing pipeline.

In addition to implementation, the project examines the behavior of the model during training and inference. This includes studying the generated speech samples and analyzing the variance adaptor's duration, pitch, and energy predictions for the prescribed test sentences.

The project therefore addresses both the technical requirements of language adaptation and the experimental evaluation of the resulting speech synthesis systems.

2.8 Objectives of the Project

The major objectives of this project are:

1. To study the theoretical foundations, architecture, and working principles of the FastSpeech 2 neural Text-to-Speech synthesis model.

2. To examine the original open-source implementation and understand its major components, dependencies, and processing pipeline.

3. To prepare the English Svarah and Telugu IndicTTS datasets for the respective training and validation experiments.

4. To perform the required audio preprocessing, text preparation, speech alignment, and acoustic feature extraction.

5. To implement and evaluate FastSpeech 2 separately for English and Telugu using appropriate language-specific configurations.

6. To extend the text-processing pipeline to support Telugu Unicode characters while preserving the existing English functionality.

7. To generate synthesized speech using the prescribed English and Telugu test sentences without modifying their original textual content.

8. To analyze the variance adaptor's duration, pitch, and energy predictions and understand their contribution to speech generation.

9. To examine training observations, generated audio samples, and relevant visualizations to assess the behavior of the implemented systems.

10. To document the implementation procedure, modifications, experimental observations, challenges, and limitations encountered during the project.

2.9 Scope of the Project

The scope of this project is restricted to the implementation and experimental evaluation of FastSpeech 2 for English and Telugu using the selected speech datasets.

The work covers the preparation of textual and acoustic data, language-specific text processing, extraction of acoustic features, model configuration, training, and speech generation.

The English and Telugu experiments are handled separately to accommodate the differences in their datasets and text representations. The Telugu implementation specifically includes modifications to support the Telugu Unicode character set within the existing text-processing pipeline.

The evaluation focuses on the prescribed test sentences, generated audio samples, training observations, and the duration, pitch, and energy predictions produced by the variance adaptor.

The project does not aim to develop an entirely new speech synthesis architecture. Instead, it focuses on adapting and evaluating an existing FastSpeech 2 implementation for two languages and documenting the technical changes required for this purpose.

2.10 Contributions of the Project

The main contributions of this project are summarized as follows:

- Implementation of the FastSpeech 2 framework for separate English and Telugu speech synthesis experiments.
- Preparation and processing of the selected English and Telugu speech datasets for the respective experiments.
- Extension of the existing text processing pipeline to support Telugu Unicode character representations.
- Preservation of the existing English text-processing functionality while introducing Telugu-specific support.
- Generation and organization of synthesized speech samples using the prescribed test sentences.
- Examination of duration, pitch, and energy predictions produced by the variance adaptor.
- Documentation of the implementation procedures, experimental outputs, and language-specific challenges.

3 Literature Review

Text-to-Speech synthesis has evolved from traditional concatenative and statistical approaches to modern deep learning-based architectures. Early neural models such as Tacotron and Tacotron 2 demonstrated the effectiveness of encoder-decoder networks for generating speech from text. However, their autoregressive generation process introduced sequential dependencies during inference. To address this limitation, FastSpeech introduced a non-autoregressive architecture that generates acoustic representations in parallel using a duration-based length regulator. FastSpeech 2 further improved this approach by incorporating ground-truth duration, pitch, and energy information through a variance adaptor, enabling more explicit modeling of speech timing and prosody. These developments provide the theoretical foundation for the present project, which implements and evaluates FastSpeech 2 separately for English and Telugu using language-specific datasets and text-processing modifications.

4 Dataset Description

The performance of a Text-to-Speech system depends significantly on the quality, quantity, and consistency of its training data. The FastSpeech 2 implementation requires paired textual and speech data from which the model learns the relationship between linguistic input and acoustic representations.

For this project, two separate datasets were selected to implement and evaluate speech synthesis for English and Telugu. The Svarah dataset was used for the English experiments, while the IndicTTS Telugu dataset was used for the Telugu experiments. Separate dataset preparation and language-specific text processing were performed for the two languages.

The dataset preparation process involved inspecting the available recordings and transcripts, selecting the required language-specific samples, validating the audio files, preparing metadata, performing speech alignment, and organizing the data into training and validation subsets.

4.1 English Dataset: Svarah

The Svarah dataset was selected for the English Text-to-Speech experiments. Svarah is a multilingual speech dataset containing recordings and corresponding transcriptions from multiple Indian languages.

An initial inspection of the available Svarah corpus identified 6,656 utterances across 19 languages, with an aggregate audio duration of approximately 9.61 hours. The inspection also identified 334 Telugu utterances. However, these Telugu recordings were not used for the Telugu experiments, since the project uses the dedicated IndicTTS Telugu dataset for that purpose.

For the English experiments, the English-labelled recordings and their corresponding transcripts were selected from Svarah. The selected samples were organized into the metadata and audio structure required by the FastSpeech 2 preprocessing pipeline.

The English dataset selection was performed with the objective of obtaining approximately five hours of training data and one hour of validation data, as specified in the project requirements. The training and validation subsets were maintained separately to prevent overlap between the two sets.

The final statistics of the selected English subset are presented in Table 4.1.

Table 4.1: English dataset statistics

Parameter	**Value**
Dataset	Svarah
Language	English
Total aligned utterances	3,399
Training utterances	2,837
Validation utterances	562
Training audio duration	4.9391 hours
Validation audio duration	0.9963 hours
Total audio duration	5.9353 hours
Audio sampling rate	22,050 Hz
Audio format	WAV
Train-validation overlap	0 utterances

4.2 Telugu Dataset: IndicTTS Telugu

The IndicTTS Telugu dataset was selected for the Telugu Text-to-Speech experiments. It provides Telugu speech recordings paired with their corresponding textual transcriptions.

The dataset was obtained in a format that required preparation before it could be used by the FastSpeech 2 preprocessing pipeline. The original dataset contained 8,576 utterances. The speech recordings and corresponding transcripts were inspected, and the required audio and metadata files were organized for subsequent processing.

The locally prepared Telugu dataset contained 1,854 WAV recordings and 1,854 corresponding metadata entries. Validation checks confirmed that there were no missing audio files, extra audio files, empty metadata entries, or duplicate entries in this prepared subset.

The dataset was subsequently processed for speech alignment. Out of the 1,854 prepared utterances, 1,846 were successfully aligned, while eight utterances were excluded from the aligned dataset.

The final training and validation organization contained 1,661 training utterances and 185 validation utterances. The corresponding audio durations were approximately 9.01 hours for training and 58.11 minutes for validation.

The resulting Telugu dataset statistics are summarized in Table 4.2.

Table 4.2: Telugu dataset statistics

Parameter	**Value**
Dataset	IndicTTS Telugu
Language	Telugu
Original dataset utterances	8,576
Locally prepared utterances	1,854
Successfully aligned utterances	1,846
Excluded utterances	8
Training utterances	1,661
Validation utterances	185
Training audio duration	Approximately 9.01 hours
Validation audio duration	Approximately 58.11 minutes
Audio format	WAV

4.3 Dataset Selection and Subsampling

The dataset selection process was carried out independently for English and Telugu. Only language-appropriate recordings and their corresponding transcripts were considered for each experiment.

For English, the required samples were selected from the English portion of the Svarah dataset. The selection was guided by the target training and validation durations specified in the assignment.

For Telugu, the dedicated IndicTTS Telugu dataset was used instead of the Telugu subset available in Svarah. A locally prepared subset of 1,854 utterances was organized for the implementation. Following data validation and alignment, eight utterances were excluded, leaving 1,846 aligned utterances for the training and validation organization.

Subsampling and data organization were intended to provide a manageable dataset while retaining the corresponding text and speech information required for acoustic model training.

The final selection statistics and duration distributions are reported separately for the two languages to make the experimental setup reproducible.

4.4 Audio and Text Preprocessing

Before training, the selected recordings and transcriptions were prepared in a format compatible with the FastSpeech 2 implementation.

The preprocessing procedure included the following operations:

1. **Audio validation:** The availability and readability of the selected WAV recordings were checked. Missing or invalid recordings were identified before further processing.

2. **Transcript validation:** Each audio recording was associated with its corresponding textual transcription. Metadata entries were inspected for empty values, duplicate entries, and missing references.

3. **Text normalization:** Text was prepared for conversion into model-compatible symbol sequences. Language-specific text representation was considered for English and Telugu.

4. **Audio standardization:** Audio recordings were prepared according to the sampling-rate and acoustic feature requirements of the FastSpeech 2 configuration.

5. **Acoustic feature extraction:** The preprocessing pipeline was used to generate the acoustic representations required by the model, including mel-spectrogram, pitch, and energy features.

6. **Metadata preparation:** The validated utterance information was organized into the metadata format expected by the training pipeline.

For Telugu, additional text-processing considerations were necessary because the language uses the Telugu Unicode writing system. A dedicated Telugu symbol inventory was introduced to represent Telugu characters and marks within the model's text-processing pipeline. The language-specific modifications are discussed in detail in Section 5.

4.5 Speech Alignment and Annotation

Speech alignment is an important preprocessing stage because FastSpeech 2 requires duration information to learn the temporal relationship between input text units and acoustic frames.

Montreal Forced Aligner (MFA) was used to generate phone-level alignment annotations for the Telugu recordings. The alignment process used the Telugu acoustic model and pronunciation dictionary prepared for the dataset.

The resulting TextGrid files contain time-aligned phone-level annotations. These annotations provide the temporal information required to derive phoneme duration targets for model training.

For the locally prepared Telugu dataset, 1,846 out of 1,854 utterances were successfully aligned. The remaining eight utterances were excluded from the aligned dataset.

The alignment outputs were subsequently used in the Telugu preprocessing workflow. The alignment procedure and language-specific implementation details are discussed further in Section 5.

4.6 Training and Validation Split

The selected recordings were organized into separate training and validation subsets. The training subset was used to optimize the model parameters, while the validation subset was used to monitor model performance on recordings not included in training.

The separation of these subsets is important for evaluating the model's ability to generalize to previously unseen utterances. No utterance was intentionally shared between the training and validation sets.

For the Telugu experiments, the final organized dataset contained 1,661 training utterances and 185 validation utterances. The corresponding durations were approximately 9.01 hours and 58.11 minutes, respectively.

For the English experiments, the split was organized according to the selected Svarah samples and the target duration requirements of the assignment. The final utterance counts and durations are reported in Table 4.1.

4.7 Dataset Summary

The project uses two distinct datasets to evaluate the FastSpeech 2 architecture across English and Telugu. Svarah provides the English speech data, while IndicTTS Telugu provides the dedicated Telugu speech data.

Although both datasets provide paired textual and audio information, their preprocessing requirements differ because of the language-specific text representations and alignment resources.

The prepared datasets form the input to the subsequent implementation, training, and evaluation stages. The dataset statistics are used to interpret the experimental results and to understand the differences observed between the English and Telugu models.

5 Methodology and Implementation

5.1 Implementation Approach

The project was implemented using an existing open-source FastSpeech 2 implementation as the reference framework. Rather than developing the architecture from scratch, the existing model components and preprocessing pipeline were retained wherever possible. The implementation was extended to support the two target languages, English and Telugu, through separate dataset preparation, text processing, and training configurations.

The overall implementation followed a sequence of data preparation, text processing, acoustic feature extraction, model training, and speech synthesis. The same fundamental FastSpeech 2 architecture was used for both languages, while language-specific components were configured independently.

The implementation can be divided into the following major stages:

1. Preparing and validating the English and Telugu speech datasets.
2. Converting textual transcriptions into model-compatible symbol sequences.
3. Extracting acoustic features and preparing duration, pitch, and energy information.
4. Configuring and training separate models for English and Telugu.
5. Generating speech from the prescribed test sentences.
6. Examining the synthesized audio and variance adaptor predictions.

5.2 FastSpeech 2 Architecture

FastSpeech 2 is a non-autoregressive neural Text-to-Speech architecture that generates mel-spectrogram representations from textual input. Its main components are the Transformer encoder, variance adaptor, and Transformer decoder.

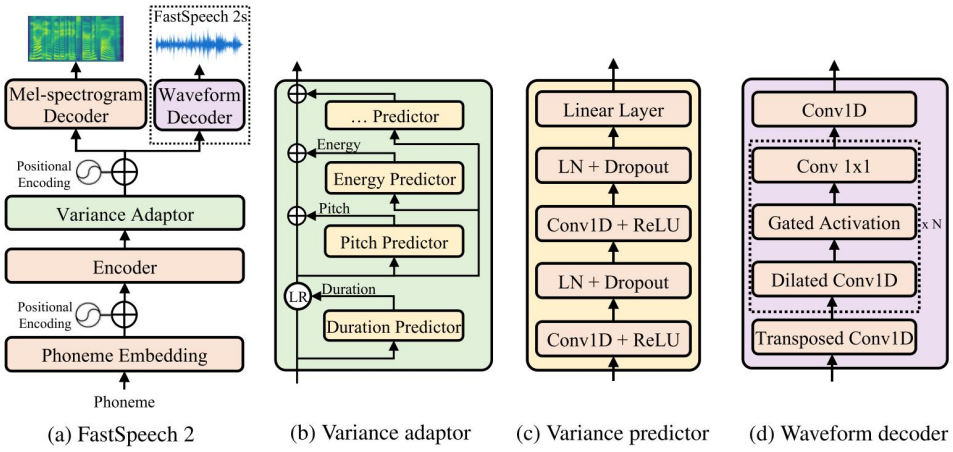


Figure 1: The overall architecture for FastSpeech 2 and 2s. LR in subfigure (b) denotes the length regulator operation proposed in FastSpeech. LN in subfigure (c) denotes layer normalization. Variance predictor represents duration/pitch/energy predictor.

Figure 5.1: Architecture of the FastSpeech 2 model

5.2.1 Transformer Encoder

The encoder receives the input text in the form of a sequence of linguistic symbols or phonemes. The input sequence is converted into embeddings and processed through Transformer layers to produce contextualized representations.

These representations capture information about the input sequence and provide the intermediate features required by the variance adaptor.

5.2.2 Variance Adaptor

The variance adaptor incorporates information about duration, pitch, and energy into the encoder representations. These features are important for modeling the temporal and prosodic characteristics of speech.

It consists of three main predictors:

- **Duration Predictor:** Estimates the duration associated with each input phoneme in acoustic frames. The predicted durations are used by the length regulator to expand the encoder representations.
- **Pitch Predictor:** Estimates fundamental frequency-related information, representing variations in the pitch of synthesized speech.
- **Energy Predictor:** Estimates energy-related information representing variations in speech intensity.

The pitch and energy representations are incorporated into the encoder outputs through their corresponding embedding mechanisms.

5.2.3 Length Regulator

The length regulator expands the encoder representations according to the duration information associated with the input phonemes. This operation produces a frame-level sequence suitable for the Transformer decoder.

It allows the model to generate acoustic frames in parallel instead of predicting them one at a time.

5.2.4 Transformer Decoder

The expanded representations are passed to the Transformer decoder, which predicts the mel-spectrogram corresponding to the input text. The mel-spectrogram represents the acoustic characteristics of the speech and is subsequently converted into an audio waveform using a compatible vocoder.

5.3 Architecture Workflow

The overall processing flow of FastSpeech 2 used in the project is summarized below.

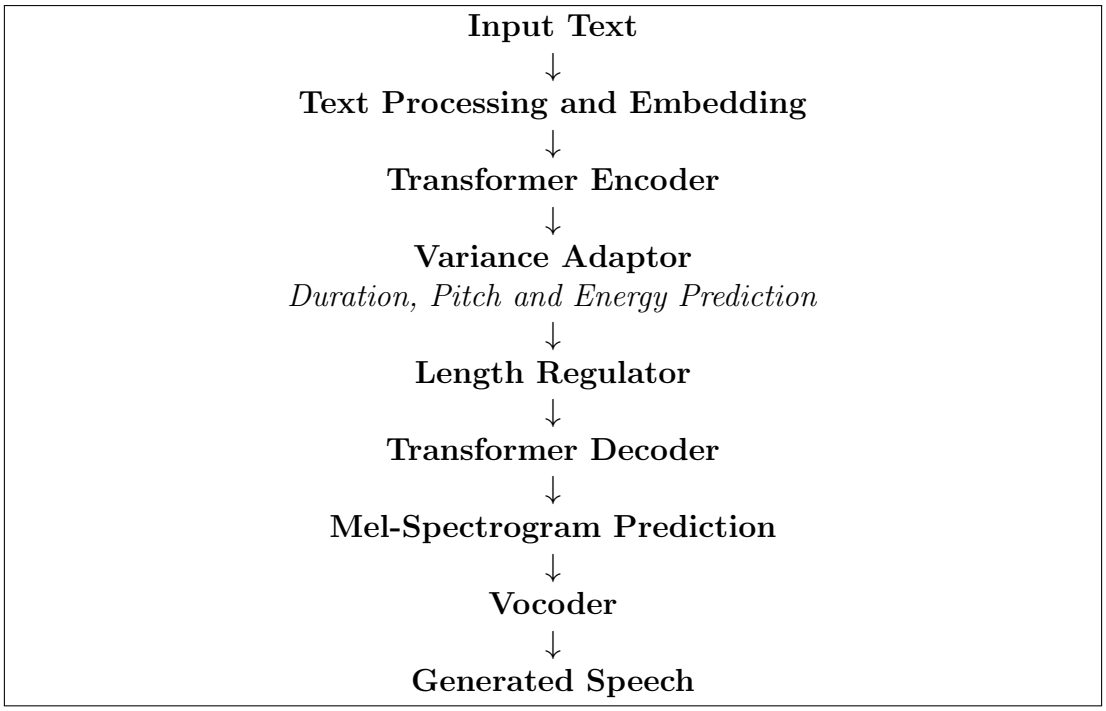


Figure 5.2: Overall FastSpeech 2 architecture workflow

5.4 Reference Implementation Components

The existing FastSpeech 2 repository provided the core model architecture and supporting training and inference components. The implementation was adapted by retaining the existing model structure and extending the language-dependent processing components.

The major components used from the reference implementation included:

- **Model architecture:** Transformer encoder, variance adaptor, length regulator, and Transformer decoder.
- **Dataset processing:** Dataset loading and batching components used to prepare textual and acoustic training inputs.
- **Training pipeline:** Model optimization, loss computation, checkpoint saving, and validation.
- **Inference pipeline:** Text processing, model loading, acoustic prediction, and speech sample generation.
- **Acoustic feature processing:** Mel-spectrogram, pitch, energy, and duration-related processing utilities.

The reference architecture was retained to maintain consistency with the original FastSpeech 2 approach. The modifications were primarily focused on language-specific text representation, dataset preparation, and experimental configuration.

5.5 English Implementation

The English implementation used the existing text-processing functionality of the reference FastSpeech 2 repository. The selected Svarah recordings and their corresponding transcripts were prepared using the English preprocessing pipeline.

The English text-processing components were retained, and the dataset was converted into the representations required by the model. The acoustic features and alignment information were prepared before training.

A separate English configuration was used to define the training parameters, dataset locations, and output directories. This allowed the English experiment to be conducted independently of the Telugu experiment.

The English model was trained using the reference architecture without requiring the introduction of a new language-specific symbol inventory.

5.6 Telugu Implementation and Modifications

Supporting Telugu required modifications to the text-processing components because the language uses the Telugu Unicode script, which is not directly covered by the original English-oriented symbol representation.

The implementation was extended to represent Telugu characters and associated marks while preserving the existing English functionality.

5.6.1 Telugu Symbol Inventory

A separate Telugu symbol inventory was introduced in the file `text/telugu_symbols.py`. It defines the padding symbol, supported punctuation, and Telugu Unicode characters and marks.

The inventory includes characters from the Telugu Unicode block, covering the range U+0C00 to U+0C4F.

This modification provides the text-processing pipeline with a language-specific set of symbols required to represent Telugu input.

5.6.2 Text Module Integration

The existing `text/__init__.py` module was modified to import the Telugu symbol definitions and expose the corresponding text-processing functionality.

This integration allows the implementation to access the Telugu symbol inventory without replacing the existing English text-processing components.

The dataset processing code continues to use the existing text conversion interface through the `text` module. This approach minimizes changes to the original dataset-loading structure.

5.6.3 Tokenizer Validation

The Telugu text-processing modifications were tested independently before proceeding with the remaining experiments.

The Telugu input string was converted into a sequence of symbol IDs and reconstructed using the corresponding reverse conversion function. The reconstructed output matched the original input.

The existing English text-processing functionality was also tested using `Hello world.` to verify that the Telugu modifications did not interfere with English processing.

These tests confirmed that the Telugu symbol representation and text conversion functions operated as expected for the tested examples.

5.6.4 Telugu Speech Alignment

The Telugu dataset required language-specific speech alignment resources. Montreal Forced Aligner (MFA) was used with the prepared Telugu acoustic model and pronunciation dictionary to generate phone-level alignment annotations.

The resulting TextGrid files provided time-aligned phone information used in the duration-processing stage.

The alignment outputs were inspected to identify the phone labels present in the prepared corpus. This inspection helped verify the alignment information used by the Telugu preprocessing pipeline.

5.7 Separate Language Configurations

The English and Telugu experiments were organized independently to prevent unintended mixing of language-specific data and outputs.

Separate configurations and output directories were maintained for each language. This organization allowed training checkpoints, logs, validation outputs, and generated speech samples to be stored separately.

The use of independent configurations also made it possible to adjust dataset paths and language-specific settings without changing the core FastSpeech 2 architecture.

5.8 Speech Synthesis and Evaluation

After training, the models were used to synthesize speech from the prescribed English and Telugu test sentences.

The inference process converts the input text into the corresponding symbol sequence, obtains acoustic predictions from the trained model, and generates the final speech waveform using the configured vocoder.

The generated samples were examined for pronunciation, intelligibility, and overall speech characteristics. In addition, the variance adaptor outputs were analyzed to understand the predicted duration, pitch, and energy patterns for individual test sentences.

The detailed experimental results and observations are presented in the subsequent sections.

6 Experimental Setup and Training Configuration

6.1 Computational Environment

The FastSpeech 2 implementation was developed using Python and the PyTorch deep learning framework. Local Windows environments were used for code development, dataset inspection, and preprocessing, while GPU-enabled cloud environments were considered for computationally intensive training.

The project used a CUDA-compatible NVIDIA Tesla T4 GPU environment during the cloud-based experiments. Montreal Forced Aligner (MFA) was used for Telugu speech alignment.

The main software and computational components are summarized in Table 6.1.

Table 6.1: Software and computational environment

Component	**Specification**
Programming language	Python 3.10.22
Deep learning framework	PyTorch
GPU used in cloud environment	NVIDIA Tesla T4
GPU memory	Approximately 14.56 GB
Speech alignment software	Montreal Forced Aligner
MFA version	3.4.3 (local environment)
Development environment	Windows and cloud notebooks

6.2 Common Acoustic Configuration

The acoustic feature extraction settings were configured to maintain consistency in the mel-spectrogram representation. The English and Telugu preprocessing configurations used a sampling rate of 22,050 Hz and 80 mel channels.

The Telugu preprocessing configuration also specified a hop length of 256, a filter length and window length of 1024, a minimum mel frequency of 0 Hz, and a maximum mel frequency of 8000 Hz.

The corresponding settings are presented in Table 6.2.

Table 6.2: Acoustic feature configuration

Parameter	Value
Sampling rate	22,050 Hz
Number of mel channels	80
Hop length	256
Filter length	1024
Window length	1024
Minimum mel frequency	0 Hz
Maximum mel frequency	8000 Hz

6.3 English Training Configuration

The English experiment used the Svarah dataset and the existing English text-processing pipeline. The training parameters were defined in the English `train.yaml` configuration file.

The model was configured for a batch size of four and a total training schedule of 50,000 steps. Training progress was recorded every 50 steps, while synthesis and validation were performed every 500 steps. Model checkpoints were saved every 1,000 steps.

The optimizer used Adam with beta parameters of 0.9 and 0.98, an epsilon value of 10^{-9}, and zero weight decay. The learning-rate warm-up was configured for 4,000 steps.

The main English training parameters are presented in Table 6.3.

Table 6.3: English FastSpeech 2 training configuration

Parameter	Value
Dataset	Svarah
Batch size	4
Total configured steps	50,000
Validation samples	649
Logging interval	50 steps
Synthesis interval	500 steps
Validation interval	500 steps
Checkpoint interval	1,000 steps

6.4 Telugu Training Configuration

The Telugu experiment used the IndicTTS Telugu dataset and the modified Telugu text-processing components.

Separate Telugu configuration files were maintained during the experiments. The main `train.yaml` configuration specified a batch size of two and a training schedule of 3,000 steps per training run. Training logs were recorded every 50 steps, validation and synthesis were configured at intervals of 1,000 steps, and checkpoints were saved every 250 steps.

An additional `train_run.yaml` configuration was also used during a recovery experiment. It specified a batch size of one, 3,000 training steps, a logging interval of 50 steps, and a checkpoint interval of 500 steps. Synthesis and validation were disabled for this run by setting their intervals to 999,999 steps.

The verified Telugu training configurations are summarized in Table 6.4.

Table 6.4: Telugu FastSpeech 2 training configurations

Parameter	`train.yaml`	`train_run.yaml`
Batch size	2	1
Total steps per run	3,000	3,000
Logging interval	50	50
Synthesis interval	1,000	999,999
Validation interval	1,000	999,999
Checkpoint interval	250	500
Validation subset size	185	–

The Telugu preprocessing configuration specified a validation subset of 185 utterances. It also used phoneme-level normalized pitch and energy representations. The language identifier was set to `te`.

6.5 Training Procedure

The training procedure was carried out independently for the English and Telugu experiments. The prepared dataset, language-specific configuration, and preprocessed acoustic features were loaded before model initialization.

The general training procedure consisted of the following stages:

1. Loading the language-specific training configuration and dataset metadata.
2. Initializing the FastSpeech 2 model and its trainable parameters.
3. Loading the textual representations and corresponding acoustic features.
4. Predicting mel-spectrogram, duration, pitch, and energy values.
5. Computing the training loss and updating model parameters through backpropagation.
6. Performing validation and synthesis at the configured intervals.
7. Saving model checkpoints and recording training logs.

The training objective consists of loss components associated with mel-spectrogram prediction, post-net mel-spectrogram prediction, duration prediction, pitch prediction, and energy prediction.

These loss components provide feedback for optimizing the different parts of the FastSpeech 2 architecture.

6.6 Training Monitoring and Checkpointing

Training progress was monitored using the loss values recorded during the optimization process. The monitored components included total loss, mel-spectrogram loss, post-net mel-spectrogram loss, duration loss, pitch loss, and energy loss.

TensorBoard event logs were used to inspect the training history and visualize changes in the individual loss components.

For the English experiment, the configured training schedule was 50,000 steps. However, the cloud runtime was interrupted before completion. The retained English backup checkpoint was the 13,000-step checkpoint. Therefore, the configured training schedule should not be interpreted as the number of steps successfully completed in the retained experiment.

For Telugu, the recovery experiment produced a checkpoint at step 27,000, stored at:

```
output/ckpt/Telugu/27000.pth.tar
```

The checkpoint and training logs were retained for subsequent synthesis and analysis.

The observed training and validation behaviour is discussed in detail in Section 7.

6.7 Inference Configuration

After training, the available model checkpoints were used to generate speech from the prescribed test sentences.

The inference process used the corresponding language configuration and text-processing components. The input text was converted into a sequence of symbols and passed through the trained model to predict acoustic representations.

The predicted mel-spectrogram was then converted into an audio waveform using the configured vocoder.

The generated speech samples and variance adaptor outputs were retained for subsequent evaluation of pronunciation, speech quality, duration, pitch, and energy.

6.8 Experimental Organization

The experiments were organized into separate language-specific directories to maintain a clear distinction between training and evaluation outputs.

The main output categories included:

- Model checkpoints.

- Training and validation logs.
- Generated speech samples.
- Mel-spectrogram and acoustic feature outputs.
- Duration, pitch, and energy prediction plots.
- Evaluation and analysis files.

This organization supports the comparison of the English and Telugu experiments and provides a structured basis for reproducing the preprocessing, training, and inference procedures.

7 Training and Validation Analysis

7.1 Loss Components

FastSpeech 2 uses multiple loss components to optimize acoustic feature prediction. During training, the model learns to predict mel-spectrogram representations along with duration, pitch, and energy information.

The training objective consists of the following loss components:

- **Mel-spectrogram loss:** Measures the difference between the predicted mel-spectrogram and the corresponding ground-truth representation.
- **Post-net mel loss:** Measures the difference between the refined mel-spectrogram produced by the post-net and the ground-truth mel-spectrogram.
- **Duration loss:** Measures the prediction error associated with the duration of phonemes or linguistic units.
- **Pitch loss:** Measures the difference between predicted and ground-truth pitch representations.
- **Energy loss:** Measures the prediction error associated with the energy of speech segments.

The total loss combines these individual objectives and provides an overall measure of the model's training error. Monitoring the individual components helps identify which aspects of acoustic and prosodic prediction contribute to the training behaviour.

7.2 English Training Analysis

The English model was trained using the Svarah dataset and the English configuration described in Section 6.

The configured training schedule consisted of 50,000 steps. During the experiment, training progressed to approximately 46,949 steps before the cloud runtime was interrupted. Although the training run progressed beyond the 13,000-step checkpoint, the retained backup available for recovery was the checkpoint saved at step 13,000.

Training logs were recorded using TensorBoard. The recorded scalar measurements included total loss, mel-spectrogram loss, post-net mel loss, duration loss, pitch loss, and energy loss. These measurements were used to inspect the model's optimization behaviour throughout the available training history.

The English training curves are used to examine the changes in the individual loss components and the overall training objective. The loss trends provide information about the model's learning progress, while the retained checkpoint determines the model state available for subsequent evaluation and recovery.

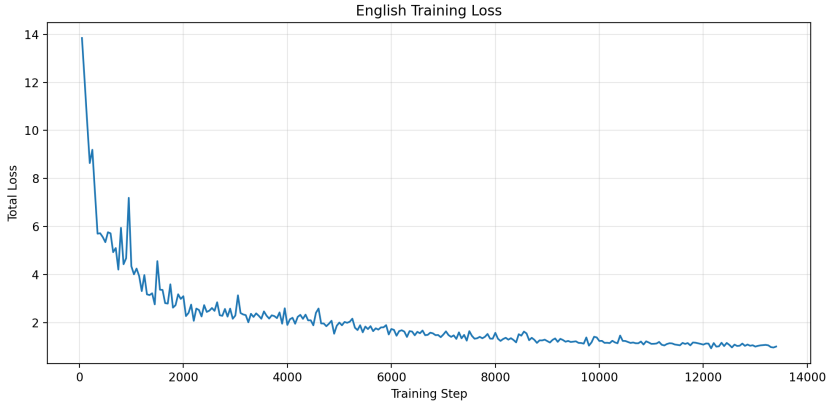


Figure 7.1: Training loss curves for the English FastSpeech 2 model



Figure 7.2: Validation loss curves for the English FastSpeech 2 model

7.3 Telugu Training Analysis

The Telugu model was trained using the prepared IndicTTS Telugu dataset and the language-specific modifications described in the methodology section.

The experiment involved an initial training configuration followed by a subsequent training run using the separate `train_run.yaml` configuration. The latter run used a batch size of one and resumed from a previously saved checkpoint.

The Telugu training logs and checkpoints were retained for further analysis. A checkpoint at step 27,000 was available under the Telugu recovery output directory.

The Telugu loss components were examined to understand the optimization behaviour of the model. In particular, the total loss and the individual mel-spectrogram, duration, pitch, and energy losses were considered when assessing the model's learning progress.

Since periodic validation was not performed during the recovery run, the training loss values from that run alone cannot establish generalization performance. The available validation results from the initial experiment and the generated speech samples provide additional information for evaluating the model.



Figure 7.3: Training loss curves for the Telugu FastSpeech 2 model

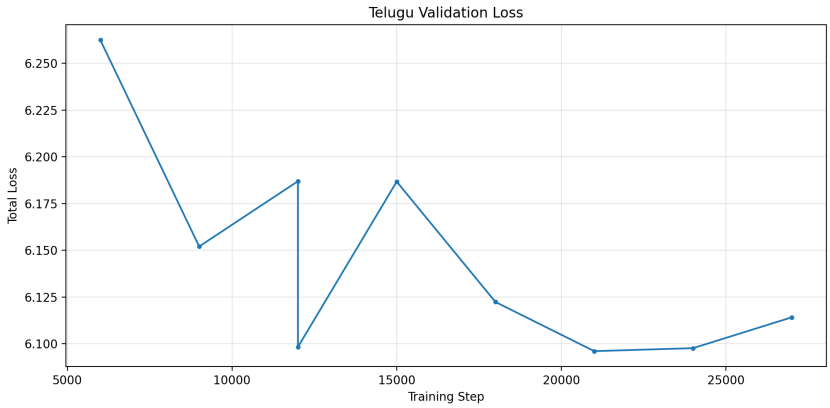


Figure 7.4: Training loss curves for the Telugu FastSpeech 2 model

7.4 Comparison of Training Progress

The English and Telugu experiments differed in their training configurations, dataset characteristics, and execution history. Therefore, their loss values should be interpreted in the context of their respective training procedures rather than compared directly as equivalent measurements.

The main training progress information is summarized in Table 7.1.

Table 7.1: Summary of training progress

Parameter	**English**	**Telugu**
Dataset	Svarah	IndicTTS Telugu
Configured steps	50,000	3,000 per run
Observed progress	Approximately 46,949 steps	Checkpoint at step 27,000
Retained checkpoint	13,000-step backup	27,000-step checkpoint
Training interruption	Cloud runtime interruption	Recovery training run

The difference between configured training steps and retained checkpoint steps is important when interpreting the results. The English experiment progressed further than the last retained backup checkpoint, whereas the Telugu experiment used a subsequent recovery run to continue training from an earlier saved state.

7.5 Representative Acoustic and Variance Adaptor Analysis

To examine the acoustic output and prosodic predictions, one representative test sample from each language was selected. The spectrograms provide a visual representation of the generated acoustic features, while the variance adaptor plots illustrate the predicted duration, pitch, and energy.

7.5.1 English Sample (E1)

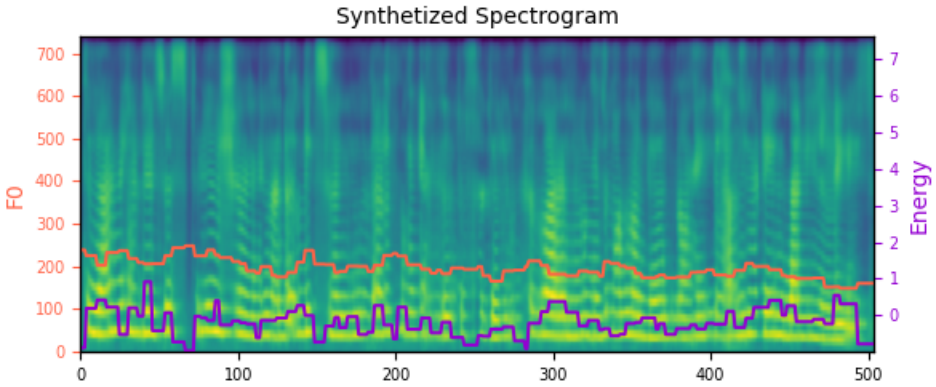


Figure 7.5: Generated mel-spectrogram for English sample E1.

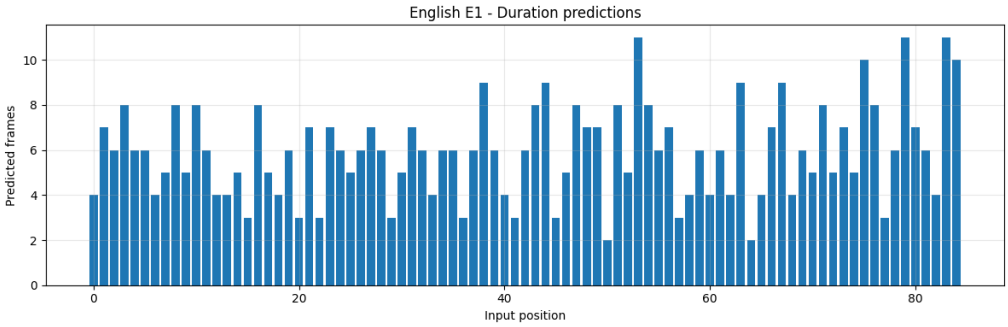


Figure 7.6: Duration prediction for English sample E1.

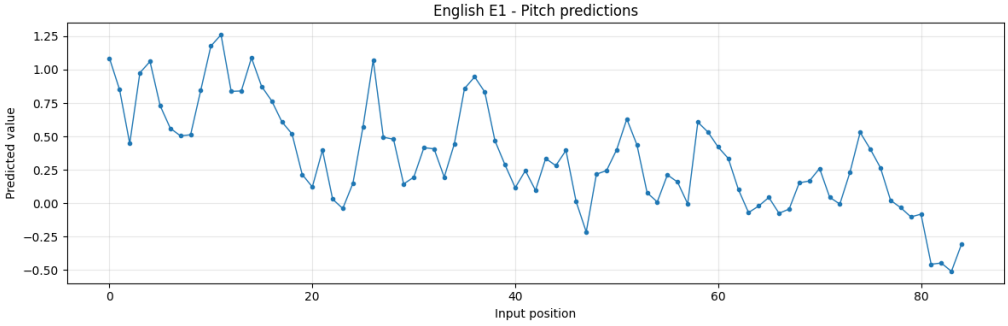


Figure 7.7: Pitch prediction for English sample E1.

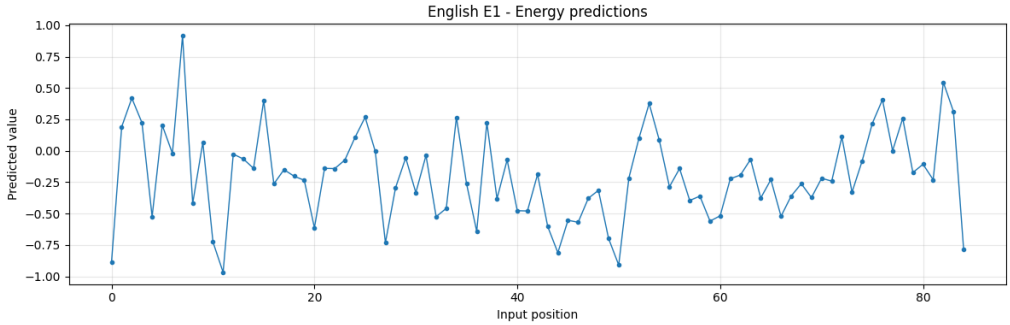


Figure 7.8: Energy prediction for English sample E1.

7.5.2 Telugu Sample (T1)

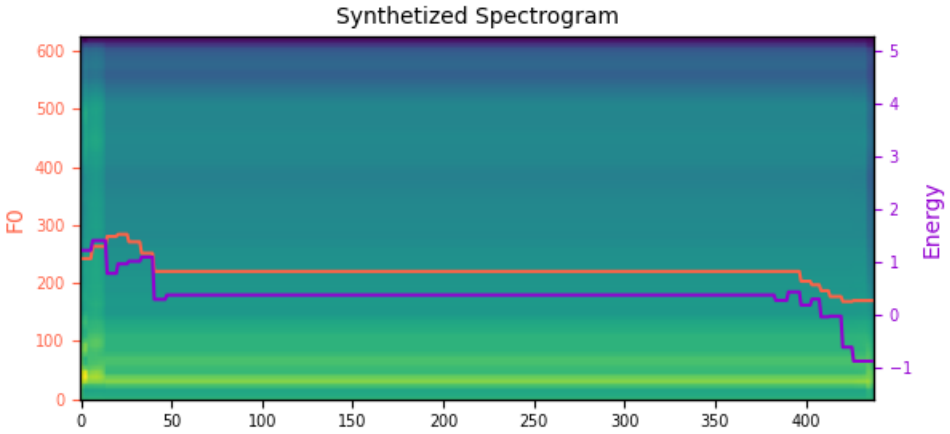


Figure 7.9: Generated mel-spectrogram for Telugu sample T1.

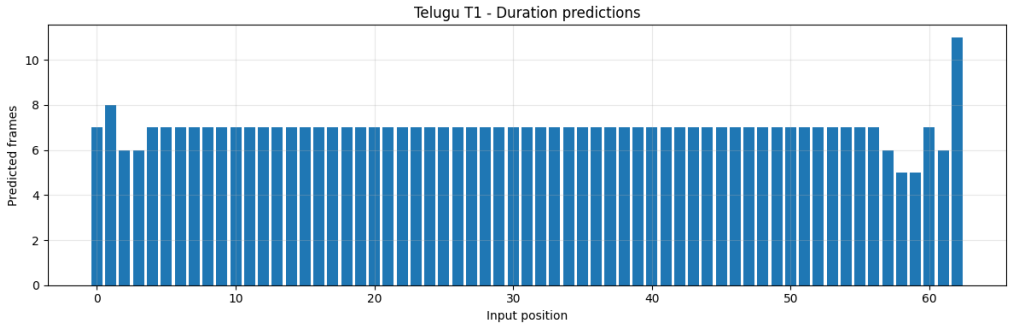


Figure 7.10: Duration prediction for Telugu sample T1.

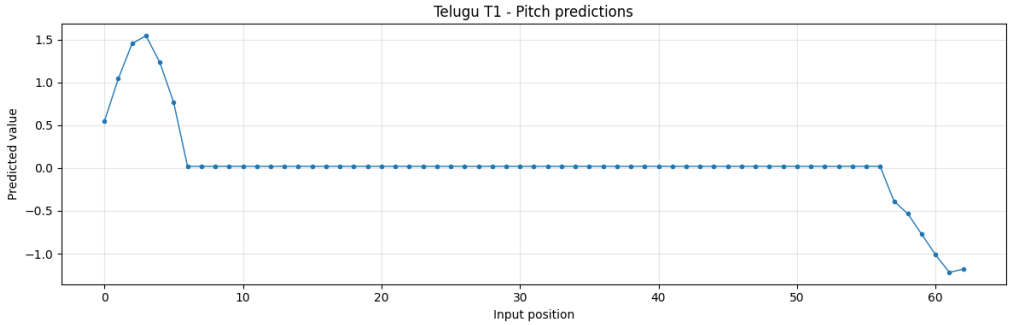


Figure 7.11: Pitch prediction for Telugu sample T1.

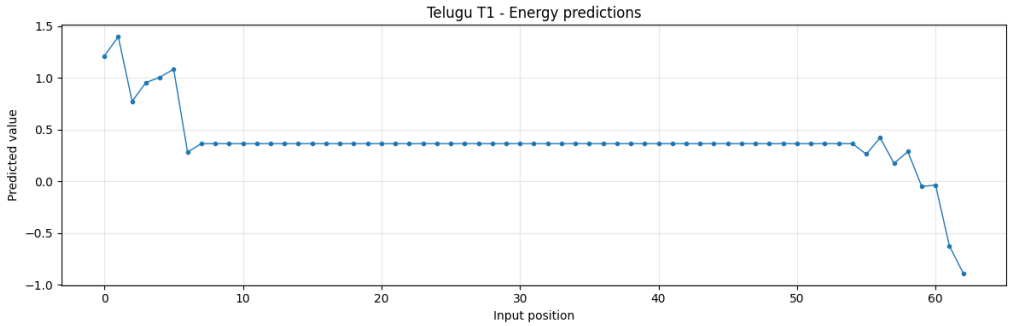


Figure 7.12: Energy prediction for Telugu sample T1.

7.6 Observations and Limitations

The training logs provide insight into the optimization process, but loss values alone are insufficient to establish the quality of generated speech. In particular, a decrease in training loss does not necessarily guarantee accurate pronunciation, natural prosody, or good speech intelligibility.

The following limitations were considered when interpreting the experiments:

- The English training run was interrupted before reaching its configured 50,000-step schedule.
- The retained English checkpoint was an earlier backup and did not represent the final state reached during the interrupted run.
- The Telugu recovery run did not perform periodic validation or synthesis because of the large configured intervals.
- Training loss and validation loss must be considered separately when assessing generalization.
- The acoustic and prosodic quality of the generated samples must also be evaluated through listening and variance adaptor analysis.

Consequently, the loss curves are interpreted together with the generated speech samples and the duration, pitch, and energy predictions discussed in the following sections.

8 Comparative Analysis and Discussion

This section compares the implementation and experimental outcomes of FastSpeech 2 for English and Telugu. Although both systems follow the same underlying architecture, their data preparation and text-processing requirements differ because of the linguistic characteristics of the two languages.

8.1 Dataset and Preprocessing Comparison

The English implementation uses the English portion of the Svarah dataset, while the Telugu implementation uses the Telugu subset of the IndicTTS corpus. Both datasets require audio-text pairing and preprocessing before they can be used for acoustic model training.

A major difference lies in text representation. English uses the original text-processing pipeline provided by the reference implementation. Telugu requires additional Unicode character support to represent its script correctly. A language-specific symbol inventory was therefore added to the implementation.

The Telugu dataset also required a separate alignment and preparation process using a Telugu acoustic model and pronunciation dictionary. These language-specific steps were necessary to prepare the duration information required by the FastSpeech 2 variance adaptor.

8.2 Implementation Comparison

The core FastSpeech 2 architecture was retained for both languages. Separate configurations and output directories were maintained to ensure that the experiments could be executed and evaluated independently.

The primary implementation difference was the text processing pipeline. English continued to use the reference implementation's existing symbols and tokenization. Telugu required the addition of Telugu Unicode symbols and corresponding language-specific handling.

The Telugu tokenizer was tested using the sentence . The converted sequence was successfully converted back to the original text. An English tokenizer test using `Hello world.` also passed after the modifications, confirming that the Telugu additions did not disrupt the existing English text-processing behavior.

8.3 Training and Evaluation Comparison

The two language models were trained using separate configurations. Their training progress and available checkpoints are discussed in Section 7.

The English experiment was configured for 50,000 training steps. However, the training run was interrupted before completion, and the available checkpoint history does not represent a completed 50,000-step experiment.

The Telugu experiment used a separate training configuration and produced checkpoints for subsequent synthesis and analysis. The training and validation observations for each language should be interpreted in the context of the corresponding configuration and the checkpoints available for evaluation.

8.4 Language-Specific Challenges

The main challenges encountered during the English implementation were related to training continuity, checkpoint preservation, and runtime availability.

For Telugu, the additional challenges involved Unicode text representation, language-specific symbol handling, and preparation of phoneme-level alignment information. The Telugu acoustic model and dictionary were used during the alignment process to obtain the annotations required for preprocessing.

These differences demonstrate that adapting a multilingual speech synthesis architecture involves more than changing the training dataset. Language-specific text processing and alignment resources are also important components of the implementation.

8.5 Discussion

The experiments demonstrate the implementation of a shared FastSpeech 2 architecture for two languages with different writing systems and preprocessing requirements. The English implementation largely follows the original reference pipeline, whereas the Telugu implementation requires additional language-specific modifications.

The comparison also highlights the importance of reliable training infrastructure and checkpoint management. Training progress alone is not sufficient to establish model quality; synthesized speech, acoustic predictions, and validation behavior must also be examined.

The final assessment of each language model is therefore based on the available synthesis samples, training and validation observations, and variance adaptor predictions presented in the preceding sections. Any limitations in training completion or output quality are considered when interpreting the results.

9 Conclusion

This project implemented and evaluated the FastSpeech 2 text-to-speech architecture for English and Telugu using the Svarah and IndicTTS datasets, respectively. The work included dataset preparation, language-specific text processing, acoustic feature extraction, model training, and speech synthesis.

The English implementation retained the reference text-processing pipeline, while the Telugu implementation required additional Unicode symbol support and language-specific alignment resources. Separate configurations were maintained for both languages.

The experiments provided an opportunity to examine training behavior, synthesized speech, and the variance adaptor's duration, pitch, and energy predictions. However, limitations in training continuity and available checkpoints must be considered when interpreting the results.

Future work can focus on completing longer training runs, improving synthesis quality, and conducting more comprehensive quantitative and perceptual evaluations.